МИНОБРНАУКИ РОССИИ

федеральное государственное бюджетное образовательное учреждение высшего образования
«Санкт-Петербургский государственный технологический институт (технический университет)»

УГНС	09.00.00	Информатика и вычислительная техника	
Направление подготовки	09.03.01	Информатика и вычислительная техника	
Направленность (профиль)		Системы автоматизированного проектирования	
Форма обучения		очная	
Факультет		Информационных технологий и управления	
Кафедра		Систем автоматизированного проектирования и управления	
Учебная дисциплина		Разработка программных систем	
Курс	II	Группа	443

Отчёт по лабораторной работе № 3

Вариант № 4

Тема: «Изучение реализации графических пользовательских интерфейсов и интеграции базы данных для веб-приложений»

Исполнитель: Чибис С.А.

Проверили: Дамрин А.О.

Кочанов С.А.

Санкт-Петербург 2026

1 Цель работы

Разработать веб-приложения для работы с массивами целых чисел, которое позволяет пользователю: вводить массив вручную, загружать из файла или генерировать случайные массивы, сортировать массивы с использованием алгоритма битонной сортировки, сохранять массивы на компьютер, просматривать результаты сортировки и управлять массивами через графический интерфейс, организовать регистрацию и авторизацию пользователей для индивидуального хранения данных.

2 Исходные данные

Разработать веб-приложение для работы с массивами целых чисел. Реализовать следующие возможности:

1. Регистрация и авторизация пользователей с логином и паролем.
2. Ввод массива пользователем с клавиатуры.
3. Загрузка массива из текстового файла .txt.
4. Генерация массива случайных целых чисел.
5. Сортировка массива алгоритмом битонной сортировки.
6. Сохранение исходного и отсортированного массива.
7. Просмотр ранее сохранённых массивов авторизованным пользователем.
8. Разработка графического интерфейса с элементами управления и отображением результатов сортировки.

3 Описание используемых технологий

Backend: Python + Flask

- **Flask (веб-фреймворк)**
- **Достоинства:**
- Минималистичный, легкий
- Отличная документация
- Гибкая маршрутизация
- Jinja2 шаблонизатор
- **Недостатки:**
- Меньше чем Django
- Требует ручной настройки БД

Frontend: HTML, CSS, JavaScript

- Обеспечивает ввод массива, генерацию случайных чисел, отображение результатов и сохранение на ПК.
- Достоинства**:** динамическая работа без перезагрузки, поддержка всех браузеров, простота реализации интерфейса.
- Недостатки**:** ручная обработка ошибок, отсутствие управления состоянием, кросс-браузерная проверка требует внимания.

База данных: SQLite

- **Достоинства:**
- Не требует сервера
- Одна файловая БД

- Поддержка JSON
- Отлично для прототипов
- **Недостатки:**
- Однопоточность
- Не для high-load

Графический интерфейс (UI/UX)

- Поля ввода, кнопки действий, отображение исходного и отсортированного массива.
- Достоинства**:** минимализм, видимость статуса системы, обработка ошибок, простота использования.
- Недостатки**:** ограниченная справка, нет адаптации под мобильные устройства, нет системы управления состоянием интерфейса.

4 Структура базы данных

Используется реляционная база данных **SQLite**, обеспечивающая хранение пользователей и истории массивов целых чисел, как исходных, так и отсортированных. Структура базы данных включает таблицу users для учёта пользователей и таблицу arrays для хранения массивов, связанных с конкретными пользователями через username.

```
bitonic_app.db
├── users (
│   id (PK),
```

```
│   username (UNIQUE),
│   password_hash,
│   created_at
│)
└── arrays (
    id (PK),
    user_id (FK),
    original_array (JSON),
    sorted_array (JSON),
    array_length (INT),
    sorting_time (REAL),
    created_at
)
```

JSON хранение массивов:

```json
{
  "original": [543, 218, 789, 123],
  "sorted": [123, 218, 543, 789]
}
```

5 Результат выполнения тестов

Для проверки работы приложения были проведены интеграционные тесты, использующие отдельную тестовую базу данных. Тесты выполнялись отдельно от основной программы и проверяли корректность работы с базой данных и алгоритмом сортировки.

```
🧪 ИНТЕГРАЦИОННЫЕ ТЕСТЫ ЛАБОРАТОРНОЙ №3
======================================================================
test_add_arrays (__main__.TestDatabaseIntegration.test_add_arrays)
✅A) Добавление 100/1000/10000 массивов ...
📤 ТЕСТ A: Добавление массивов
✅100 массивов: 0.28с ✓
✅1000 массивов: 3.11с ✓
✅10000 массивов: 32.23с ✓
ok
test_cleanup (__main__.TestDatabaseIntegration.test_cleanup)
✅B) Очистка БД ...
🧹 ТЕСТ C: Очистка БД
🧹 100 записей (#1): 0.0000с ✓
🧹 100 записей (#2): 0.0000с ✓
🧹 100 записей (#3): 0.0156с ✓
🧹 1000 записей (#1): 0.0000с ✓
🧹 1000 записей (#2): 0.0000с ✓
🧹 1000 записей (#3): 0.0156с ✓
ok
test_load_and_sort (__main__.TestDatabaseIntegration.test_load_and_sort)
✅Б) Выгрузка + сортировка ...
📤 ТЕСТ Б: Выгрузка и сортировка
📊 Запуск #1: 0.00с, среднее: 0.000000с ✓
📊 Запуск #2: 0.00с, среднее: 0.000000с ✓
📊 Запуск #3: 0.00с, среднее: 0.000000с ✓
ok

----------------------------------------------------------------------
Ran 3 tests in 42.301s
```

Рисунок 1 – результат выполнения работы разработанной программы

6 Описание графического интерфейса и соответствие эвристикам UI/UX

Интерфейс выполнен как веб-страница с HTML, CSS и JavaScript. Верхняя панель отображает имя пользователя и кнопку «Выйти» (видимость

статуса системы). Пользователь может вводить массив вручную, загружать из файла или генерировать случайный.

Кнопки «Сортировать», «Скачать», расположены согласованно, что обеспечивает управляемость. Интерфейс минималистичен, ошибки ввода и недопустимые имена файлов обрабатываются и отображаются рядом с полями, что помогает пользователю и обеспечивает обратную связь.

Эвристики UI/UX, реализованные в интерфейсе: видимость состояния системы, соответствие действиям пользователя, управляемость, минимализм, обратная связь, помощь пользователю в выявлении ошибок.

7 Вывод

Разработано веб-приложение с регистрацией, авторизацией, вводом, генерацией, загрузкой, сортировкой и сохранением массивов. Все функции работают корректно, данные пользователей и массивы надежно сохраняются в базе данных. Интеграционные тесты подтвердили стабильность работы алгоритма битонной сортировки и корректность взаимодействия с базой данных.